

CS4555 Compiler Construction: Group 1 - Report

Andreas Tsatsanis, Konstantin Mirinski, Panagiotis Panagi

January 2026

1 Introduction

For our project for the Compiler Construction course, we are tasked with designing an unambiguous language of our choice that generates programs containing variable and function definitions and declarations, arithmetic and boolean expressions, conditionals, assignments, and loops. Then, we are responsible for implementing a parser for these programs, creating their Control Flow Graph (CFG), and performing one out of the four proposed data flow analyses. In our case, we selected to perform an Available Expressions (AEs) analysis as we deemed this to be the most intriguing option that would offer a challenge that would help us understand better the purpose of each of our compiler passes and how compilers can be further optimized.

In general, this analysis defines AEs as those that at a specific program point, are computed in every control flow path before reaching it without having any of the variables they depend on be altered in the meantime; thus, these expressions do not need to be recomputed [1]. Specifically, this is a forward data analysis, meaning that we ought to follow the normal control flow from start to finish and propagate any identified available expressions at a specific point to its subsequent ones. Then, we can traverse the sets of AEs at every instruction and check if that instruction recomputes something that is contained in these sets.

Nonetheless, this report stands to explain our thought process behind implementing this project in Rust and the nuances that come from it. Consequently, Section 2 will discuss the important design decisions and main components of our project, Section 3 will outline its limitations, and Section 4 will reflect on its quality and the workload distribution among the team members, and conclude the report.

2 Project Overview

2.1 Language Definition

To start off our project, we define a language and grammar that generates all accepted programs. Our target language is inspired by Rust itself in both how it is formulated and in how it is expression-centric. Syntactic similarities of these two include that all expressions, declarations, and assignments end with a semicolon (“;”), that functions, if, and while statements need their bodies wrapped in curly braces (“{,}” - which we will later call “blocks”), that the later two’s conditions do not need to be wrapped in parenthesis, that variable declarations occur with the “let” keyword, that comments are wrapped between either “/*”-“*\” or “//”-“\n”, that whitespace and indentations are ignored, and that the last expression of a block is assumed to be its return value and does not need to end in a semicolon. This allowed us to easily create mock programs based on this grammar to test our Rust project without having to think in a language that is vastly different from our implementation language.

Semantically, an inputted program is expected to abide by the following: A program contains at least one function. A function comprises a set of parameters (identifiers followed by a type and separated with commas) and an expression body. This body is usually a block but can be a simple arithmetic expression too, for example. That is because blocks contain zero or more statements, possibly followed by an expression that is not proceeded with a semicolon (the block’s return value). On the contrary, statements include anything that has to be followed by a semicolon, whether that is a variable declaration, an assignment, or an expression. As discussed perviously, decelerations start with the “let” keyword and are followed by an identifier, its type, and an assignment to the identifiers initial expression value, while assignments are similar but do not require the “let” keyword or a type declaration.

Nonetheless, this all comes down to expressions: As expected, these are any combinations of and, or, and xor on equalities, which compare (using the standard comparison operators) arithmetic expressions of addition, subtraction, multiplication, and division on unary expressions. These refer to raising something to an exponent or negating an arithmetic expression. Still, these are applied to “primaries” - the lowest grammatical form in our language. For example, they refer to integers, booleans, identifiers, or function calls. Additionally, they can refer to if statements, while loops, blocks, or parenthesized expressions, thus forming the recursive definition of the grammar. Importantly, separating expressions and statements, allows users to add, for example, integers with if statements, but not integers with other declarations or assignments, as those are segregated in a higher level.

All in all, this allows for a strongly typed, imperative target language that recognizes three types (integers, booleans, and units; the equivalent to Rust’s “()” type) and natively supports xor operations, exponentiation, recursion, and code block assignments.

2.2 Parsing

We implemented our grammar using Pest v.2.8.4 [2] which allowed us to use its build-in parser to get an initial parse-tree of `pest::Pairs`. Then, we passed this through our custom parser (in `ast.rs`) in order to create an AST of a `Program` by recursively traversing the `Pair` tree and building the different syntactic elements of the language, including top-level functions, parameters, types, statements, and expressions, while preserving their source code spans. These are using for more precise error reporting, but are also kept in the AST itself, as will be explained in Section 2.5. Thus, we wrap all expressions in `Exprs` which contain an expression as defined by our grammar and its `start` and `end` position in the source code. Importantly, these positions are not used for any equality or hash operations on `Expr` as those depend only on their inner expressions. Thus, all functions, statements, etc. are mapped onto `Functions`, `Statements`, etc. which rely on these `Exprs` to construct everything in a hierarchical, operator-precedence- and type-aware manner.

Clearly, there are many benefits to utilizing Pest this way. Namely, Pest grammars are unambiguous by design as they rely on a single `.pest` file (in `grammar.pest`), which is easily integrable to a Rust codebase as it can be parsed by Pest’s default parser at compile time. Additionally, Pest is simpler to use compared to other alternatives, it provides in-depth error messages that can be customized and propagated to its result, and handles rule backtracking automatically which is crucial for our recursive grammar. Thus, Pest boost’s the project’s readability and maintainability by reducing its required complexity and providing a clear blueprint on how to expand its offered parsing with our own implementations.

2.3 Uniquify

Next, we pass this AST to our Uniquify implementation in order to ensure that all variable and function names are unique. Here, we decided to break the program's namespaces into two scopes, one for all function names and one for all variables, thus allowing these two to possibly share names. This is possible due to the structure of function calls being inherently different from variables, meaning that our subsequent passes will never confuse the two and, thus, this will never affect the soundness of the program. Additionally, we allow users to define a set of pre-established functions (such as the ones that would be included in the standard library) so that the Uniquify pass loads these into the function context before running, meaning that any unbounded calls to these will not yield errors. For example, our `reserved.rs` pre-defines functions such as `print` and `println`, and the parser makes sure to reject redefining any reserved function names. Overall, both of these decisions were taken to mimic a realistic compiler as much as possible.

Nonetheless, the implementation of Uniquify itself is simple: After initializing the function and variable contexts and the global counter, the pass iterates through every function, uniquifies their name, and recursively traverses their body to uniquify their expressions. Every time an unseen identifier is reached, the pass appends to it the global counter and updates its respective context mapping to indicate this change so that all subsequent appearances of this identifier get replaced correctly. Importantly, when encountering a block, the pass pushes a new `BTreeMap` onto the variables context's vector and pops it when exiting, thus treating it like a stack. This ensures that all local variable uses refer to their closest invocation, while function definitions are global - an integral part of maintaining the program's integrity.

2.4 CFG Construction

Moving on, the CFG construction pass of our pipeline is responsible for creating a graph of `AnnotatedExpressions` from our `Program` AST. These wrap the aforementioned `Exprs` and add two fields, the `depends_on` and `mutates` set. For an assignment or deceleration, the former set contains all the expressions read during the computation of its right-hand side, while the variable on the left-hand side is contained in the latter set. This allows us to reduce any `Statement` into its most crucial components for an AE analysis: its right-hand side `Expr`, the variables it depends on, and the variable it overwrites. Thus, the complexity of the AE pass is drastically reduced as it no longer needs to extract these itself, at the cost of widening the scope of the CFG pass which still has access to the left-hand side variables it needs to cut.

Utilizing these structs, this pass sorts the list of functions comprising the target program so that the `main` function appears first. This means that it will be processed first, ensuring that the AE analysis pass can assume that it can start its forward analysis from the CFG's first node without losing any control flow paths. When the pass encounters any function, it creates an empty start and end node and maps these to the function's unique name so that any inferences made towards that function can be resolved quickly and new edges can be drawn easily - without affecting the AE analysis. In the end, by recursively traversing the structure of each `AnnotatedExpression`'s `Expr`, the pass generates subgraphs for each subexpression and combines them with the main CFG, resulting in what is seen in Figure 1, which can be generated via `Graphviz` from the `Dot` file the CFG can be stored in.

2.5 Available Expressions Analysis

Now, as we are working with `AvailableExpressions` that know which variables they read and write to, we pass the CFG to the AE analysis pass. This is implemented following the basic principles introduced in the course, assignment description, and relevant sources [1], [3], [4].

```

def is_greater_than_three(n: Int): Bool :=
{
  if n > 3 then {
    true
  } else {
    false
  }
}

def main(): Int :=
{
  let i: Int = 1;
  let cond: Bool = true;
  while cond do {
    cond = is_greater_than_three(i);
    i = i + 1;
  };
  i
}

```

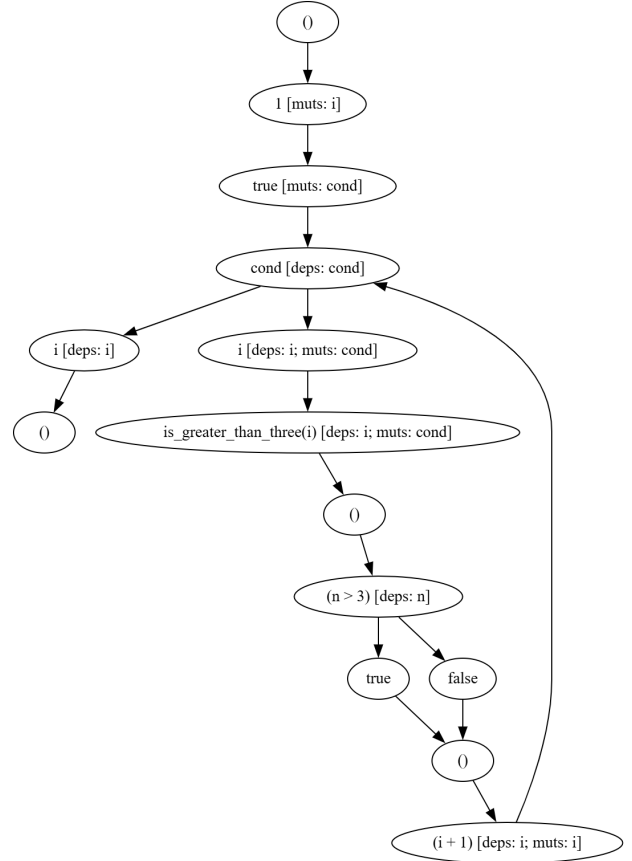


Figure 1: The resulting CFG of this pass for a simple program.

However, it is important to note that we handle traversing our CFG via the Work-list approach [4], which means that we start from the first node (assumed to be `main`'s first instruction) and for each node we examine which expressions are currently available, generated, or killed and only if we observe a change in these sets do we enqueue the node's successors as points of re-examination. This is a more efficient way of detecting AEs than the standard Fixpoint-iteration approach, where one would iterate over the entire CFG until all sets stabilize.

To find the AEs, we first need to define the three aforementioned sets of expressions that are taken into consideration at every point [3]: Firstly, the `in_set` defines the intersection of all sets of AEs that were propagated to the current point from its predecessors. This is initialized as the empty set in the beginning and will be useful when working with branching CFGs. Secondly, the `gen_set` is the set of expressions calculated by the current instruction. This is either the empty set for untrackable expressions or the singleton containing the current node's expression. Thirdly, the `kill_set` contains all the expressions from the union of these two that depend on at least one variable that the current instruction rewrites. Thus, these are no longer available and should not be considered as such moving forward. In the end, we propagate the following set to the following instruction: $(\text{in_set} \cup \text{gen_set}) - \text{kill_set}$. By performing the union before the difference, we ensure that assignments depending on the variable they are mutating are not considered available after computation as that instance of the target variable becomes stale after the assignment.

In the end, once we have identified a stable state for all the sets, we traverse the CFG once again and check if the current instruction at each point uses an expression from its `in_set`. Thus, we collect all of these `Exprs` and return an appropriate mapping with the spans each is located in. This is then used to print into standard output (see Figure 2) - and can be further visualized via the language server setup that is discussed in the project's README file to show all parsing/passes' errors and more.

3 Limitations

As a result of some of our design decisions, our tool has some innate limitations to what programs it can accept, even though we believe that this still led to a project faithful to the assignment’s guidelines. Firstly, as mentioned in Section 2.1, we assume that all programs have at least one function defined. Specifically, we assume that every program has a function named `main`, the name of which Uniquify does not alter at all, since components such as our language’s interpreter or the CFG pass, work by first identifying the program’s `main` function to establish the start of its control flow; something that is important when testing our project. Secondly, we mentioned that our language is strongly-typed, meaning that we do not allow any function, parameter, or variable declarations to not include their corresponding type. Thirdly, our statement definitions always assume that a declared variable is initialized with some expression, meaning that we reject statements such as “`let x:Int;`” if they appear anywhere inside the function’s body. Lastly, we assume that all variables are inherently mutable (thus, not supporting the distinction of explicit mutables and constants), that all parameters are passed by value (thus, not supporting an indication for “by-reference” arguments), and that when a block is assigned (e.g. to variable), that block gets executed at definition before moving on.

4 Conclusion & Reflection

Throughout this project we faced multiple challenges that we believe are noteworthy. Specifically, we had to revise our target language and grammar multiple times to make sure that it is robust, captures the minimal requirements of the assignment, and is as clear, unambiguous, and simple as possible. Moreover, the visualization of the results of our control flow and EA analysis passes proved to be an interesting endeavor, as we had to get familiarized with `Dot` and with setting up a language server, respectively. Consequently, that meant that we had to carry information about each expression’s position in the source code throughout the pipeline, which required defining custom equality, display, and hash operations for `Exprs`, `Statements`, and `AnnotatedExpressions` so that they would not affect the logic of our passes.

Given more time, there are a few aspects of the project we would have liked to expand upon. Firstly, we would expand our language definition to cover the limitations described in Section 3, which include allowing uninitialized variable declarations, implicit type definitions, programs without a main function, and functions with side-effects. Furthermore, even though we have added extensive unit and integration tests for all components in our pipeline, we would still like to expand upon them to cover as many possible use cases as possible. Lastly, even though Section 2.5 describes two ways of visualizing the results of our AE analysis, we acknowledge that the language server idea could have been expanded into an accessible extension for a popular IDE such as `VSCode` that would highlight any available expressions in the open file’s source code. Ultimately, we decided against this and implemented a custom language server setup via `neovim` instead, as this would require too much resources and effort for this project. However, if we had more time, it would be a good opportunity to attempt this anyway.

Overall the workload distribution among the team members is as follows: Andreas was responsible for setting up the project, implementing the parser pass, and visualizing the final results via the language server, Konstantin was responsible for implementing the pipeline’s CFG construction pass and annotating its nodes with their read/written variable sets, and Panagiotis was responsible for the uniquify and data flow analysis pass and formulating the report - while everyone contributed to testing and reviewing the program. Overall, we believe that despite our project’s limitations, it serves as a adequate implementation of the project’s requirements and that it serves its purpose in identifying the inputted program’s available expressions.

```

Program Annotations:

def main(): Int := {
  let a: Int = 1;
  let b: Int = 2;
  let e: Int = {
    let y: Int = a;
    let x: Int = a+b;
    x
  };
  let b: Int = a+b;
  b
}

```

Figure 2: Example output of highlighted available expressions.

References

- [1] M. Pradel, “Program analysis - Lecture 5: Data flow analysis (part2),” *Software Lab, University of Stuttgart*, 2019. [Online]. Available: https://software-lab.org/teaching/winter2019/pa/lecture_data_flow_analysis2.pdf.
- [2] Pest Team, “Pest. The elegant parser,” *Docks.rs*, 2025. [Online]. Available: <https://docs.rs/pest/2.8.4/pest/>.
- [3] T. Jones, “Lecture 4: Available expression analysis,” *University of Cambridge: Department of Computer Science and Technology*, 2019. [Online]. Available: <https://www.cl.cam.ac.uk/teaching/1920/OptComp/slides/lecture04.pdf>.
- [4] Michel Schinz, “Dataflow analysis,” *EPFL*, 2008. [Online]. Available: https://lampwww.epfl.ch/teaching/archive/advanced_compiler/2008/resources/slides/acc-2008-10-dataflow-analysis_6.pdf.